

Atividade Linux

rascunho de respostas (enquanto não aprendo como fazer a entrega no GitHub)

1

```
mkdir oficina-linux  
cd oficina-linux  
mkdir anotações  
mkdir roteiros  
mkdir testes  
ls
```

Respostas:

- O que mkdir faz? Cria pastas e diretórios
- O que a opção -p faz? Cria pastas em cascata.
- O que as chaves {} fazem no mkdir? Servem para criar várias pastas ao mesmo tempo.
- O que pwd mostra? Mostra o caminho completo de onde você tá.

2

```
cd anotações  
touch duvidas.txt  
touch comandos.txt  
echo "OIIEEEEE" > aula.txt  
cat aula.txt  
cp aula.txt ../testes/  
cd ../testes  
ls  
mv aula.txt "copia aula 1.txt"  
ls  
pwd
```

Respostas:

- Qual é a diferença entre touch, echo, cp, e mv?
- touch cria arquivo vazio;
echo exibe texto (ou salva se usado com >);
cp duplica (copia);
mv move ou renomeia.
- O que > faz?

Redireciona a saída. Em vez de mostrar o texto na tela, ele joga o conteúdo para dentro de um arquivo .

- O que aconteceu com o arquivo copiado?

Ele permaneceu no lugar original e uma cópia exata surgiu no destino.

3

```
cd ~/oficina-linux
```

```
mkdir scripts
```

```
cd scripts
```

```
ls ../anotações
```

```
cat ../anotações/aula.txt
```

```
cat ~/oficina-linux/anotações/aula.txt
```

Respostas:

- Qual é a diferença entre caminho absoluto e relativo?

O absoluto começa da raiz (/) e funciona de qualquer lugar.

O relativo parte de onde você ta agora.

- O que .. significa? Significa "a pasta acima" da que você ta agora.

- Por que /home/...e home/... não são a mesma coisa?

O primeiro (com /) busca a pasta "home" na raiz do sistema.

O segundo busca uma pasta "home" que esteja dentro da pasta onde você esta no momento.

4

```
echo '#!/bin/bash
```

```
echo "olá!"
```

```
echo "usuário atual: $(whoami)"
```

```
echo "diretório atual: $(pwd)"
```

```
echo "data e hora: $(date)" > saudacao.sh
```

```
./saudacao.sh
```

****Obs: Acesso será negado**

```
chmod +x saudacao.sh
```

```
./saudacao.sh
```

Respostas:

- O que é um script? É um arquivo de texto com vários comandos que o Linux executa em sequência, como uma "receita".
- Para que serve a linha #!/bin/bash? "Shebang". Diz ao sistema que ele deve usar o interpretador Bash para ler aquele arquivo.
- Por que o script não é executado sem permissão adequada?
Por segurança, o Linux cria arquivos sem permissão de execução para evitar que vírus ou comandos errados rodem sozinhos.
- O que chmod +x faz? Adiciona a permissão de execução ao arquivo.

5

whoami

id

groups

ls -l saudacao.sh

touch segredo.txt

chmod 600 segredo.txt

ls -l segredo.txt

Respostas:

- O que significa 600? Significa que o dono pode ler e escrever (6), e ninguém mais pode fazer nada (00)
- Quem pode ler ou alterar esse arquivo agora? Apenas o dono do arquivo e o superusuário (root).
- Qual é a diferença entre tarefas de leitura, escrita e execução?
Leitura (r) permite ver o conteúdo; Escrita (w) permite alterar; Execução (x) permite rodar o arquivo como um programa.

6

ls /root

sudo ls /root

Respostas:

- O que aconteceu sem sudo? O sistema geralmente nega os comandos que alteram arquivos vitais por segurança.
- O que mudou com sudo? Poder de executar o comando como "Superusuário"
- O que sudo você transforma em root permanentemente? O sudo te dá poderes apenas para aquele comando específico. Para ser root permanente, usa-se sudo su.

7

```
sleep 300 &  
ps aux | grep sleep  
kill [PID]
```

Respostas:

- O que é um processo? É programa que está "vivo" (sendo executado na memória agora).
- Qual é a diferença entre programa e processo?
O programa é o arquivo parado no HD. O processo é o programa em ação.
- O que o comando kill faz? Encerra um processo à força (como o "Finalizar Tarefa" do Windows).

8

```
sudo apt update  
apt search tree  
sudo apt install tree  
cd ~/oficina-linux  
tree
```

Respostas:

- O que é um gerenciador de pacotes? "Loja de aplicativos" do terminal
- O que é um pacote? Arquivo que tem o programa compactado e pronto pra instalar
- O que apt update faz? Atualiza a lista de programas disponíveis, pra saber se tem versões novas.
- O que tree você mostrou sobre sua estrutura de diretórios? Desenha a estrutura de pastas em formato de "árvore", mostrando o que tá dentro do quê.

9

```
cat << 'EOF' > hello.sh
#!/usr/bin/env bash
echo "ooooi"
echo "user: $(whoami)"
echo "directory: $(pwd)"
echo "date: $(date)"
EOF
chmod +x hello.sh
./hello.sh
```

10

```
echo '#!/bin/bash'
nome="SeuNome"
agora=$(date)
echo "Olá $nome, hoje é $agora" > variaveis.sh
chmod +x variaveis.sh
./variaveis.sh
```

11

```
echo '#!/bin/bash'
echo "Qual é seu nome?"
read nome
echo "Qual é sua idade?"
read idade
echo "Olá $nome! Você tem $idade anos e já é Team Linux!" > entrada.sh
chmod +x entrada.sh
./entrada.sh
```

12

```
echo '#!/bin/bash'
nome=$1
echo "Olá, $nome! Seja bem-vindo à oficina." > argumentos.sh
chmod +x argumentos.sh
./argumentos.sh SeuNome
```

13

```
echo '#!/bin/bash'
usuario=$(whoami)
if [ "$usuario" == "root" ]; then
    echo "Você é o superusuário (root). Cuidado com seus poderes!"
else
    echo "Você é o usuário $usuario. Você ã tem poderes de root agora."
fi > condicao.sh
chmod +x condicao.sh
./condicao.sh
sudo ./condicao.sh
```

14

```
echo '#!/bin/bash'
arquivo=$1
if [ -f "$arquivo" ]; then
    echo "O arquivo $arquivo existe!"
    if [ -r "$arquivo" ]; then
        echo "Você tem permissão para lê-lo."
    else
        echo "Você NÃO tem permissão de leitura."
    fi
else
    echo "Erro: O arquivo $arquivo ã foi encontrado."
fi > arquivos.sh
```

```
chmod +x arquivos.sh
./arquivos.sh hello.sh
./arquivos.sh segredo.txt
./arquivos.sh fantasma.txt
```

15 e 16

```
echo '#!/bin/bash'
contador=0
for arquivo in ../anotações/*.txt; do
    echo "Encontrei o arquivo: $arquivo"
    contador=$((contador + 1))
done
echo "O total de arquivos é: $contador" > loop.sh
chmod +x loop.sh
./loop.sh
```

17

```
echo '#!/bin/bash'
nome_pasta="backup-hoje"
mkdir $nome_pasta
cp ../anotações/*.txt $nome_pasta/
echo "Arquivos copiados para a pasta $nome_pasta!" > backup.sh
chmod +x backup.sh
./backup.sh
```

18

```
echo '#!/bin/bash'
cp inexistente.txt pasta_qualquer
if [ $? -eq 0 ]; then
    echo "O comando funcionou!"
else
    echo "O comando falhou! O arquivo não existe." > erro.sh
chmod +x erro.sh
./erro.sh
```

19

```
echo '#!/bin/bash'
echo "--- MEU RELATORIO ---" > relatorio.txt
echo "Usuario: $(whoami)" >> relatorio.txt
echo "Pasta: $(pwd)" >> relatorio.txt
echo "Data: $(date)" >> relatorio.txt
echo "Relatório criado no arquivo relatorio.txt!" > relatorio.sh
chmod +x relatorio.sh
./relatorio.sh
```

DESAFIO